

ECE/CS 757 Advanced Computer Architecture II

Spring 2025

Midterm Exam 2 Review

Instructor: Prof. Joshua San Miguel

Transactional Memory

The following program runs on a TM system with eager VM and eager CD. Assume there's a bug such that on a store, conflict detection neglects to check for conflicts in other threads' read sets. Which of the following program outcomes are possible? Select all that apply.

- i. A=0, B=0
- ii. A=0, B=1
- iii. A=1, B=0
- iv. A=1, B=1

```
// init
*x = 0; *y = 0;
```

```
// thread 1
atomic {
    *x = 1;
    *y = 1;
}
```

```
// thread 2
atomic {
    A = *x;
    B = *y;
}
```

Transactional Memory

The following program runs on a TM system with eager VM and eager CD. Assume there's a bug such that on a store, conflict detection neglects to check for conflicts in other threads' read sets. Which of the following program outcomes are possible? Select all that apply.

i. **A=0, B=0**

ii. A=0, B=1

iii. A=1, B=0

iv. A=1, B=1

```
A = *x;  
B = *y;  
T2 commit;  
↓  
*x = 1;  
*y = 1;
```

```
// init  
*x = 0; *y = 0;
```

```
// thread 1  
atomic {  
    *x = 1;  
    *y = 1;  
}
```

```
// thread 2  
atomic {  
    A = *x;  
    B = *y;  
}
```



Transactional Memory

The following program runs on a TM system with eager VM and eager CD. Assume there's a bug such that on a store, conflict detection neglects to check for conflicts in other threads' read sets. Which of the following program outcomes are possible? Select all that apply.

- i. **A=0, B=0**
- ii. A=0, B=1
- iii. A=1, B=0
- iv. A=1, B=1

```
// init
*x = 0; *y = 0;
```

```
// thread 1
atomic {
    *x = 1;
    *y = 1;
}
```

```
// thread 2
atomic {
    A = *x;
    B = *y;
}
```

Transactional Memory

The following program runs on a TM system with eager VM and eager CD. Assume there's a bug such that on a store, conflict detection neglects to check for conflicts in other threads' read sets. Which of the following program outcomes are possible? Select all that apply.

i. **A=0, B=0**

ii. A=0, B=1

iii. A=1, B=0

iv. A=1, B=1

```
A = *x;  
*x = 1;  
*y = 1;  
T1 commit;  
↓  
B = *y;
```

```
// init  
*x = 0; *y = 0;
```

```
// thread 1  
atomic {  
    *x = 1;  
    *y = 1;  
}
```

```
// thread 2  
atomic {  
    A = *x;  
    B = *y;  
}
```

Transactional Memory

The following program runs on a TM system with eager VM and eager CD. Assume there's a bug such that on a store, conflict detection neglects to check for conflicts in other threads' read sets. Which of the following program outcomes are possible? Select all that apply.

i. **A=0, B=0**

ii. **A=0, B=1**

iii. A=1, B=0

iv. A=1, B=1

A = *x;
*x = 1;
*y = 1;
T1 commit;
B = *y;

conflict not detected

```
// init
*x = 0; *y = 0;
```

```
// thread 1
atomic {
    *x = 1;
    *y = 1;
}
```

```
// thread 2
atomic {
    A = *x;
    B = *y;
}
```



Transactional Memory

The following program runs on a TM system with eager VM and eager CD. Assume there's a bug such that on a store, conflict detection neglects to check for conflicts in other threads' read sets. Which of the following program outcomes are possible? Select all that apply.

i. **A=0, B=0**

ii. **A=0, B=1**

iii. A=1, B=0

iv. A=1, B=1

```
// init
*x = 0; *y = 0;
```

```
// thread 1
atomic {
    *x = 1;
    *y = 1;
}
```

```
// thread 2
atomic {
    A = *x;
    B = *y;
}
```



Transactional Memory

The following program runs on a TM system with eager VM and eager CD. Assume there's a bug such that on a store, conflict detection neglects to check for conflicts in other threads' read sets. Which of the following program outcomes are possible? Select all that apply.

i. **A=0, B=0**

ii. **A=0, B=1**

iii. A=1, B=0

iv. A=1, B=1

```
*x = 1;  
A = *x;  
B = *y;  
T2 commit;  
↓  
*y = 1;
```

```
// init  
*x = 0; *y = 0;
```

```
// thread 1  
atomic {  
    *x = 1;  
    *y = 1;  
}
```

```
// thread 2  
atomic {  
    A = *x;  
    B = *y;  
}
```



Transactional Memory

The following program runs on a TM system with eager VM and eager CD. Assume there's a bug such that on a store, conflict detection neglects to check for conflicts in other threads' read sets. Which of the following program outcomes are possible? Select all that apply.

i. **A=0, B=0**

ii. **A=0, B=1**

iii. ~~A=1, B=0~~

iv. A=1, B=1

conflict is detected

```
*x = 1;  
A = *x;  
B = *y;  
T2 commit;  
*y = 1;
```

```
// init  
*x = 0; *y = 0;
```

```
// thread 1  
atomic {  
    *x = 1;  
    *y = 1;  
}
```

```
// thread 2  
atomic {  
    A = *x;  
    B = *y;  
}
```

Transactional Memory

The following program runs on a TM system with eager VM and eager CD. Assume there's a bug such that on a store, conflict detection neglects to check for conflicts in other threads' read sets. Which of the following program outcomes are possible? Select all that apply.

i. **A=0, B=0**

ii. **A=0, B=1**

iii. ~~A=1, B=0~~

iv. A=1, B=1

```
// init
*x = 0; *y = 0;
```

```
// thread 1
atomic {
    *x = 1;
    *y = 1;
}
```

```
// thread 2
atomic {
    A = *x;
    B = *y;
}
```



Transactional Memory

The following program runs on a TM system with eager VM and eager CD. Assume there's a bug such that on a store, conflict detection neglects to check for conflicts in other threads' read sets. Which of the following program outcomes are possible? Select all that apply.

i. **A=0, B=0**

ii. **A=0, B=1**

iii. ~~A=1, B=0~~

iv. **A=1, B=1**

```
*x = 1;  
*y = 1;  
T1 commit;  
A = *x;  
B = *y;
```

```
// init  
*x = 0; *y = 0;
```

```
// thread 1  
atomic {  
    *x = 1;  
    *y = 1;  
}
```

```
// thread 2  
atomic {  
    A = *x;  
    B = *y;  
}
```



Transactional Memory

Consider the following program running on a TM system. Should we use eager or lazy VM? Eager or lazy CD? Justify your answer.

```
// thread 1
atomic {
    *x = 1;
    *y = 1;
    for(int i = 0; i < 1024; i++)
        b[i] = a[i];
    *z = 1;
}
```

```
// thread 2
atomic {
    *z = 2;
    for(int i = 0; i < a[1023]; i++)
        *c += 4;
    if (a[0] == 0) *y = 0;
    else *y = 2;
    *x = 2;
}
```

Transactional Memory

Consider the following program running on a TM system. Should we use eager or lazy VM? Eager or lazy CD? Justify your answer.

```
// thread 1
atomic {
    *x = 1;
    *y = 1;
    for(int i = 0; i < 1024; i++)
        b[i] = a[i];
    *z = 1;
}
read = { a[0:1023] }
write = { x, y, z, b[0:1023] }
```

```
// thread 2
atomic {
    *z = 2;
    for(int i = 0; i < a[1023]; i++)
        *c += 4;
    if (a[0] == 0) *y = 0;
    else *y = 2;
    *x = 2;
}
read = { c, a[0], a[1023] }
write = { x, y, z, c }
```



Transactional Memory

Consider the following program running on a TM system. Should we use eager or lazy VM? Eager or lazy CD? Justify your answer.

Lazy VM: no rollbacks (since only write-write).

```
// thread 1
atomic {
    *x = 1;
    *y = 1;
    for(int i = 0; i < 1024; i++)
        b[i] = a[i];
    *z = 1;
}
read = { a[0:1023] }
write = { x, y, z, b[0:1023] }
```

```
// thread 2
atomic {
    *z = 2;
    for(int i = 0; i < a[1023]; i++)
        *c += 4;
    if (a[0] == 0) *y = 0;
    else *y = 2;
    *x = 2;
}
read = { c, a[0], a[1023] }
write = { x, y, z, c }
```



Transactional Memory

Consider the following program running on a TM system. Should we use eager or lazy VM? Eager or lazy CD? Justify your answer.

Lazy VM: no rollbacks (since only write-write). Lazy CD: only check c once.

```
// thread 1
atomic {
    *x = 1;
    *y = 1;
    for(int i = 0; i < 1024; i++)
        b[i] = a[i];
    *z = 1;
}
read = { a[0:1023] }
write = { x, y, z, b[0:1023] }
```

```
// thread 2
atomic {
    *z = 2;
    for(int i = 0; i < a[1023]; i++)
        *c += 4;
    if (a[0] == 0) *y = 0;
    else *y = 2;
    *x = 2;
}
read = { c, a[0], a[1023] }
write = { x, y, z, c }
```



Consistency

The following program runs on a processor with sequential consistency. Which of the following outcomes are possible? Select all that apply.

- | | | | | |
|------|----------|--|--|--|
| i. | A=0, B=0 | | | |
| ii. | A=0, B=1 | | | |
| iii. | A=1, B=0 | | | |
| iv. | A=1, B=1 | | | |
-
- | | | | |
|--|--------------------------------------|------------------------------|------------------------------|
| | <code>// init</code> | | |
| | <code>*x = 0; *y = 0; *z = 0;</code> | | |
| | | | |
| | <code>// P1</code> | <code>// P2</code> | <code>// P3</code> |
| | <code>*y = 1;</code> | <code>while(*y == 0);</code> | <code>while(*z == 0);</code> |
| | <code>*x = 1;</code> | <code>A = *x;</code> | <code>B = *x;</code> |
| | | <code>*z = 1;</code> | |

Consistency

The following program runs on a processor with sequential consistency. Which of the following outcomes are possible? Select all that apply.

i. **A=0, B=0**

ii. A=0, B=1

iii. A=1, B=0

iv. A=1, B=1

```
// init
```

```
*x = 0; *y = 0; *z = 0;
```

```
// P1
```

```
*y = 1;
```

```
*x = 1;
```

last

```
// P2
```

```
while(*y == 0);
```

```
A = *x;
```

```
*z = 1;
```

```
// P3
```

```
while(*z == 0);
```

```
B = *x;
```

Consistency

The following program runs on a processor with sequential consistency. Which of the following outcomes are possible? Select all that apply.

i. **A=0, B=0**

ii. A=0, B=1

iii. A=1, B=0

iv. A=1, B=1

```
// init
```

```
*x = 0; *y = 0; *z = 0;
```

```
// P1
```

```
*y = 1;
```

```
*x = 1;
```

```
// P2
```

```
while(*y == 0);
```

```
A = *x;
```

```
*z = 1;
```

```
// P3
```

```
while(*z == 0);
```

```
B = *x;
```

Consistency

The following program runs on a processor with sequential consistency. Which of the following outcomes are possible? Select all that apply.

i. **A=0, B=0**

ii. **A=0, B=1**

iii. A=1, B=0

iv. A=1, B=1

```
// init
```

```
*x = 0; *y = 0; *z = 0;
```

```
// P1
```

```
*y = 1;
```

```
*x = 1;
```

second last

```
// P2
```

```
while(*y == 0);
```

```
A = *x;
```

```
*z = 1;
```

```
// P3
```

```
while(*z == 0);
```

```
B = *x;
```

last



Consistency

The following program runs on a processor with sequential consistency. Which of the following outcomes are possible? Select all that apply.

i. **A=0, B=0**

ii. **A=0, B=1**

iii. A=1, B=0

iv. A=1, B=1

```
// init
```

```
*x = 0; *y = 0; *z = 0;
```

```
// P1
```

```
*y = 1;
```

```
*x = 1;
```

```
// P2
```

```
while(*y == 0);
```

```
A = *x;
```

```
*z = 1;
```

```
// P3
```

```
while(*z == 0);
```

```
B = *x;
```



Consistency

The following program runs on a processor with sequential consistency. Which of the following outcomes are possible? Select all that apply.

i. **A=0, B=0**

ii. **A=0, B=1**

iii. ~~A=1, B=0~~

iv. A=1, B=1

```
// init
```

```
*x = 0; *y = 0; *z = 0;
```

```
// P1
```

```
*y = 1;
```

```
*x = 1;
```

```
// P2
```

```
while(*y == 0);
```

```
A = *x;
```

```
*z = 1;
```

```
// P3
```

```
while(*z == 0);
```

```
B = *x;
```

RAW

RAW

WAR



Consistency

The following program runs on a processor with sequential consistency. Which of the following outcomes are possible? Select all that apply.

i. **A=0, B=0**

ii. **A=0, B=1**

~~iii. A=1, B=0~~

iv. A=1, B=1

```
// init
```

```
*x = 0; *y = 0; *z = 0;
```

```
// P1
```

```
*y = 1;
```

```
*x = 1;
```

```
// P2
```

```
while(*y == 0);
```

```
A = *x;
```

```
*z = 1;
```

```
// P3
```

```
while(*z == 0);
```

```
B = *x;
```



Consistency

The following program runs on a processor with sequential consistency. Which of the following outcomes are possible? Select all that apply.

i. **A=0, B=0**

ii. **A=0, B=1**

~~iii. A=1, B=0~~

iv. **A=1, B=1**

```
// init
```

```
*x = 0; *y = 0; *z = 0;
```

```
// P1
```

```
*y = 1;
```

```
*x = 1;
```

first

```
// P2
```

```
while(*y == 0);
```

```
A = *x;
```

```
*z = 1;
```

```
// P3
```

```
while(*z == 0);
```

```
B = *x;
```



Consistency

Is (A=1,B=0) possible with program ordering but without store atomicity?

```
// init
```

```
*x = 0; *y = 0; *z = 0;
```

```
// P1
```

```
*y = 1;
```

```
*x = 1;
```

```
// P2
```

```
while(*y == 0);
```

```
A = *x;
```

```
*z = 1;
```

```
// P3
```

```
while(*z == 0);
```

```
B = *x;
```

Consistency

Is (A=1,B=0) possible with program ordering but without store atomicity?

```
// init
```

```
*x = 0; *y = 0; *z = 0;
```

```
// P1
```

```
*y = 1;
```

```
*x = 1;
```

```
// P2
```

```
while(*y == 0);
```

```
A = *x;
```

```
*z = 1;
```

```
// P3
```

```
while(*z == 0);
```

```
B = *x;
```

**without store atomicity, writes
from different threads can be
observed in different orders**

Consistency

Is (A=1,B=0) possible with program ordering but without store atomicity?

```
// init  
*x = 0; *y = 0; *z = 0;
```

```
// P1  
*y = 1;  
*x = 1;
```

without store atomicity, writes
from different threads can be
observed in different orders

```
// P2  
while(*y == 0);  
A = *x;  
*z = 1;
```

without store atomicity:

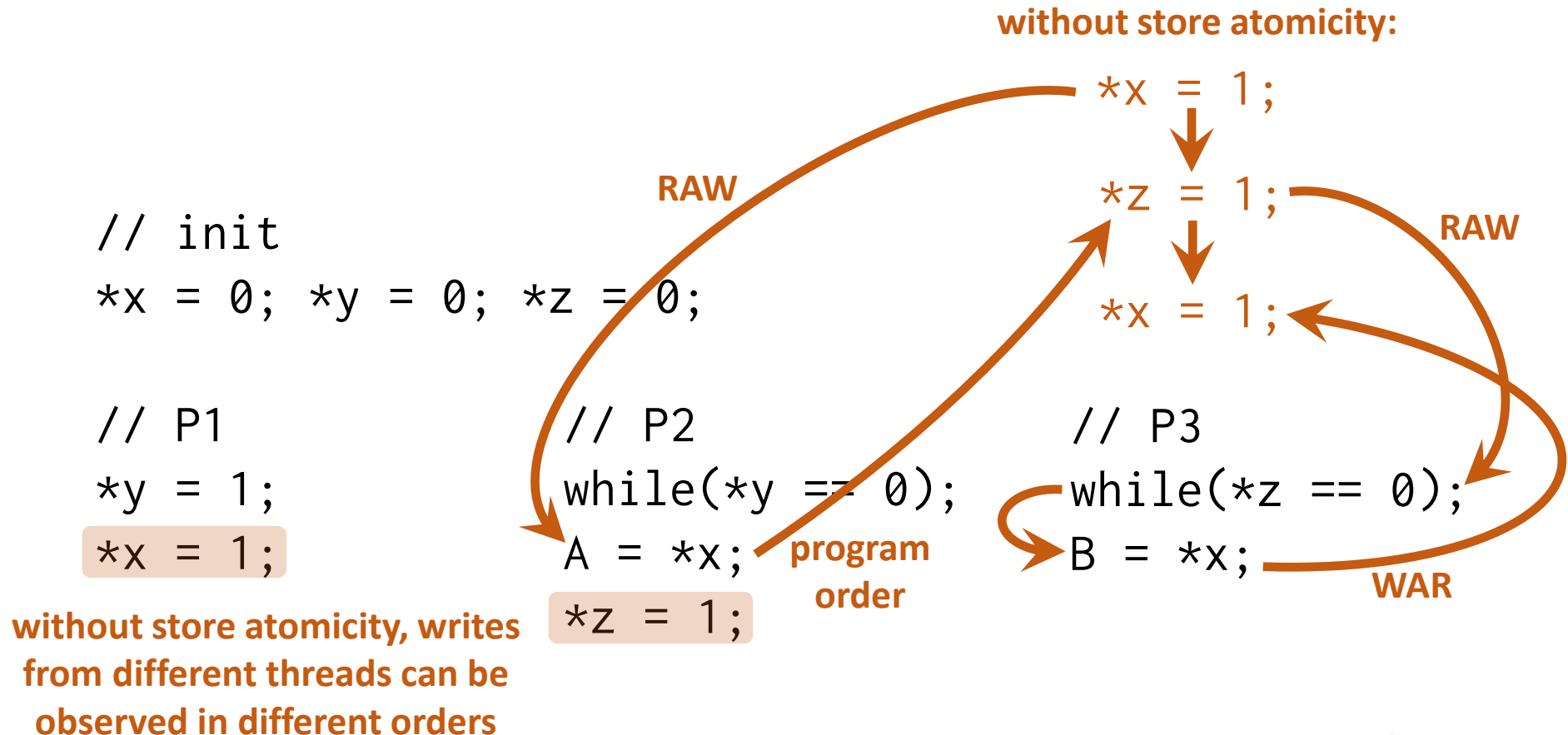
```
*x = 1;  
↓  
*z = 1;  
↓  
*x = 1;
```

```
// P3  
while(*z == 0);  
B = *x;
```

Consistency

Is (A=1,B=0) possible with program ordering but without store atomicity?

Yes



Consistency

Is (A=2,B=0) possible with sequential consistency?

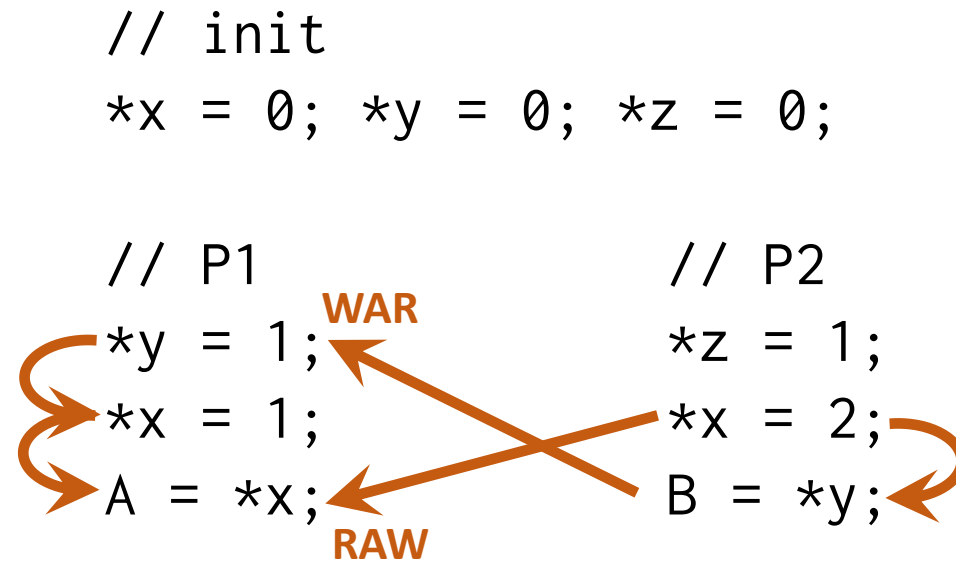
```
// init
*x = 0; *y = 0; *z = 0;
```

```
// P1
*y = 1;
*x = 1;
A = *x;
```

```
// P2
*z = 1;
*x = 2;
B = *y;
```

Consistency

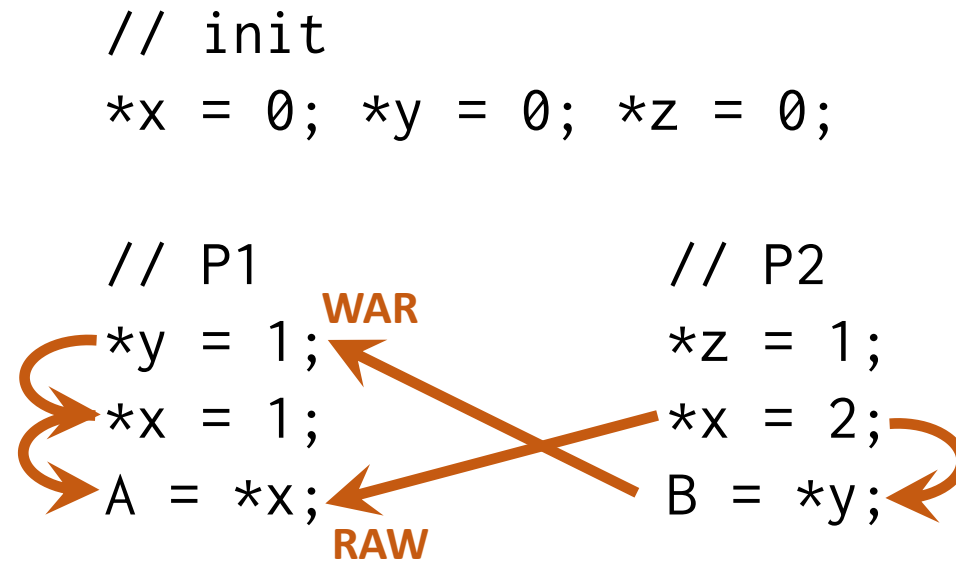
Is (A=2,B=0) possible with sequential consistency?



Consistency

Is (A=2,B=0) possible with sequential consistency?

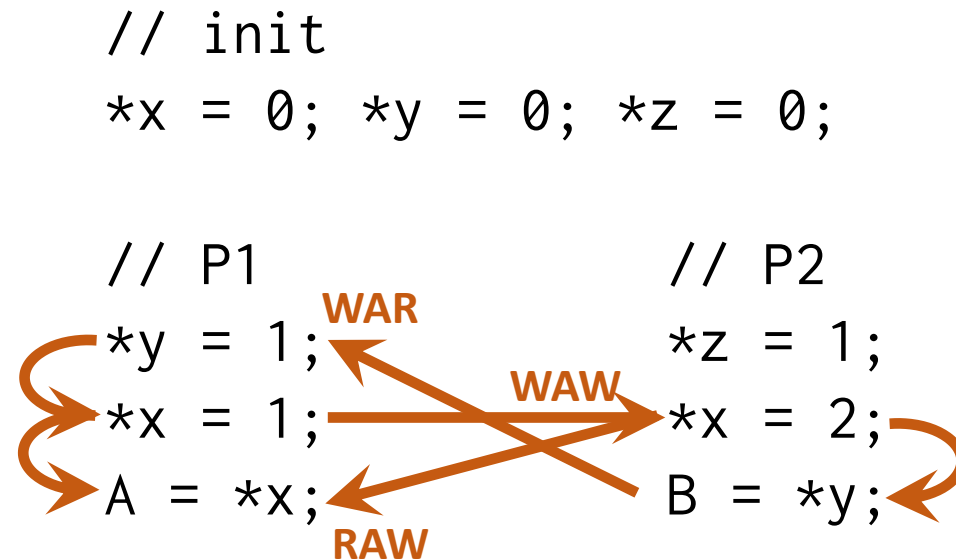
Yes?



Consistency

Is (A=2,B=0) possible with sequential consistency?

No



Interconnects

You are building a 72-node network-on-chip with a torus topology. If your goal is to minimize the maximum channel load under uniform random traffic, which one of the following topologies would you choose? Explain why.

- i. 2,6,6-ary 3-cube
- ii. 3,3,8-ary 3-cube
- iii. 8,9-ary 2-cube

Interconnects

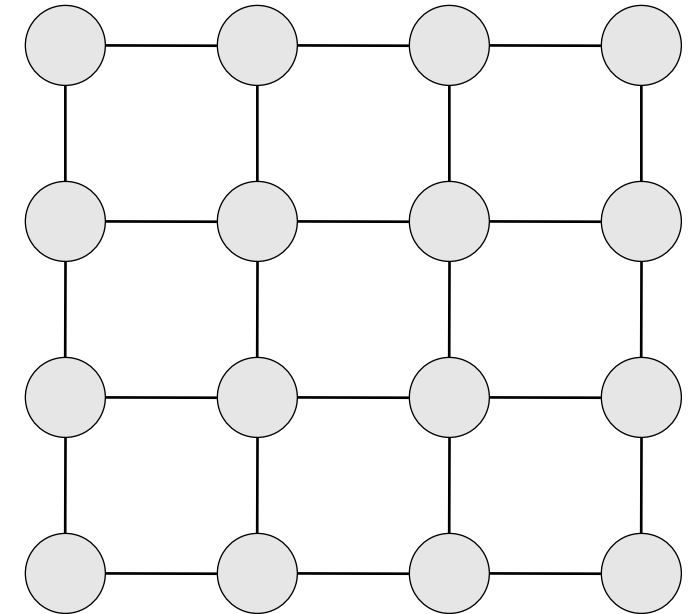
You are building a 72-node network-on-chip with a torus topology. If your goal is to minimize the maximum channel load under uniform random traffic, which one of the following topologies would you choose? Explain why.

- i. 2,6,6-ary 3-cube
- ii. 3,3,8-ary 3-cube
- iii. 8,9-ary 2-cube

Topology (i). Max channel load for a torus is at the minimum cross-cut bisection, which has bandwidth of $4N/k$, where k is the largest dimension and N is the total number of nodes (72). Topology (i) has the highest minimum cross-cut bandwidth and thus lowest maximum channel load.

Interconnects

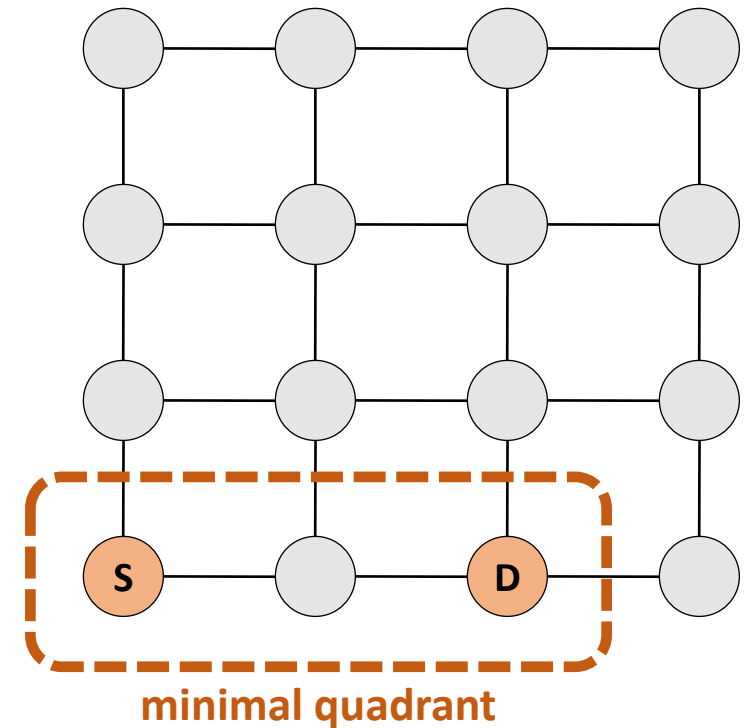
Consider a 4-ary 2-mesh NoC with minimal oblivious (Valiant's) routing. Describe an example traffic pattern where the network would perform no better than with dimension order routing.



Interconnects

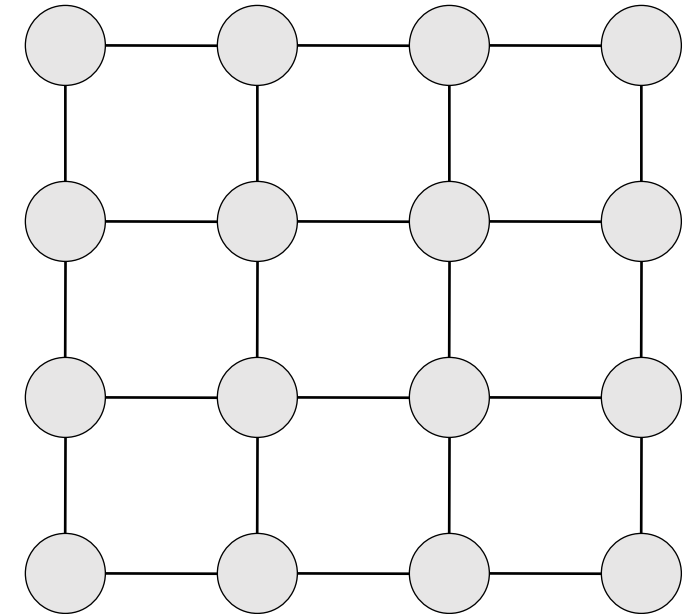
Consider a 4-ary 2-mesh NoC with minimal oblivious (Valiant's) routing. Describe an example traffic pattern where the network would perform no better than with dimension order routing.

A traffic pattern where all source-destination pairs are in either the same row or in the same column. This implies that all packets have at most one minimal path.



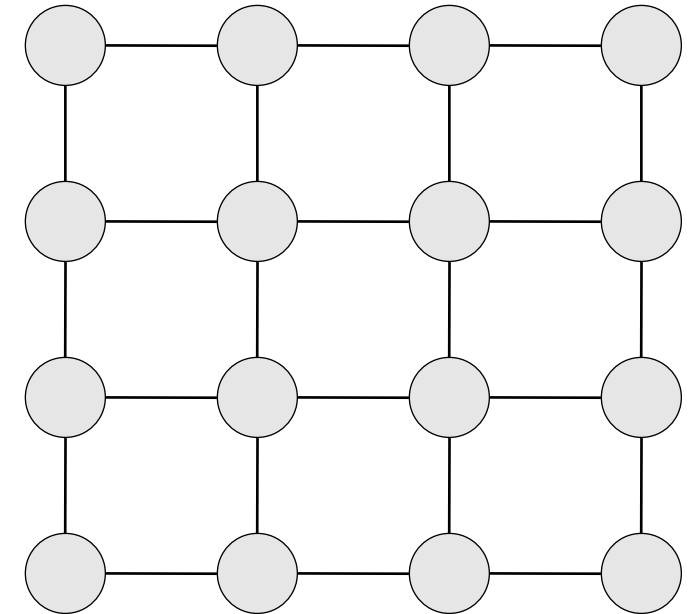
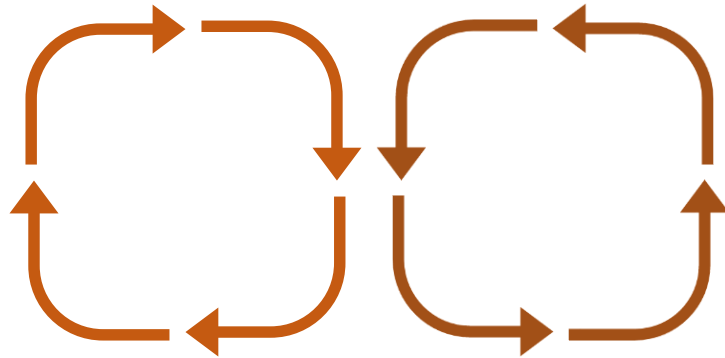
Interconnects

Consider a 4-ary 2-mesh NoC. You plan to use adaptive routing (no U-turns) with some turn restrictions to ensure deadlock-freedom. Would it be fine to disallow the following turns: E to S and S to E?



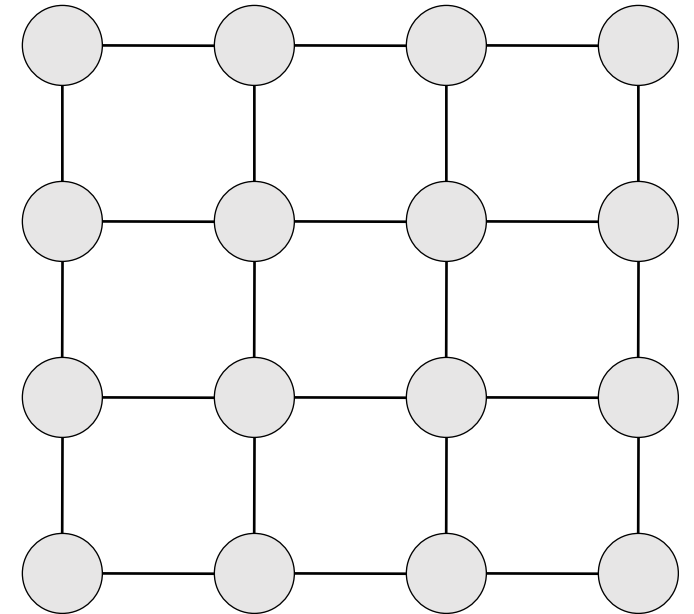
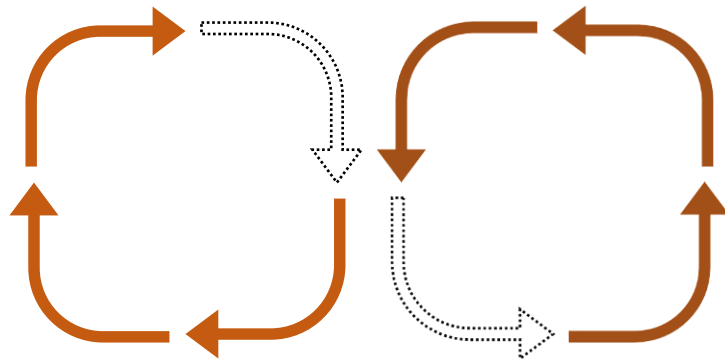
Interconnects

Consider a 4-ary 2-mesh NoC. You plan to use adaptive routing (no U-turns) with some turn restrictions to ensure deadlock-freedom. Would it be fine to disallow the following turns: E to S and S to E?



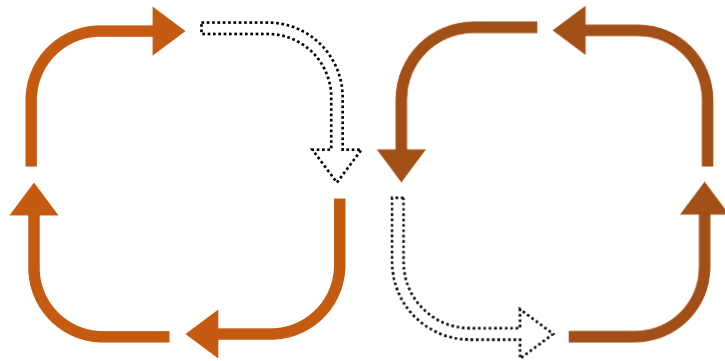
Interconnects

Consider a 4-ary 2-mesh NoC. You plan to use adaptive routing (no U-turns) with some turn restrictions to ensure deadlock-freedom. Would it be fine to disallow the following turns: E to S and S to E?

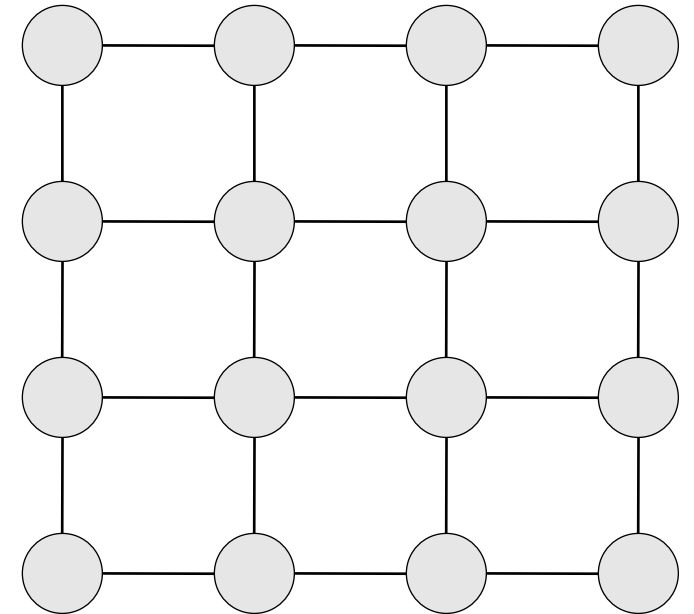


Interconnects

Consider a 4-ary 2-mesh NoC. You plan to use adaptive routing (no U-turns) with some turn restrictions to ensure deadlock-freedom. Would it be fine to disallow the following turns: E to S and S to E?

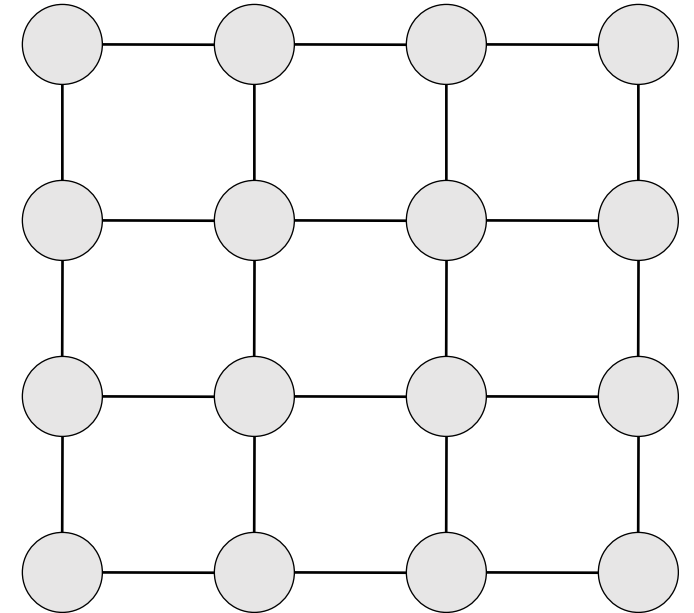
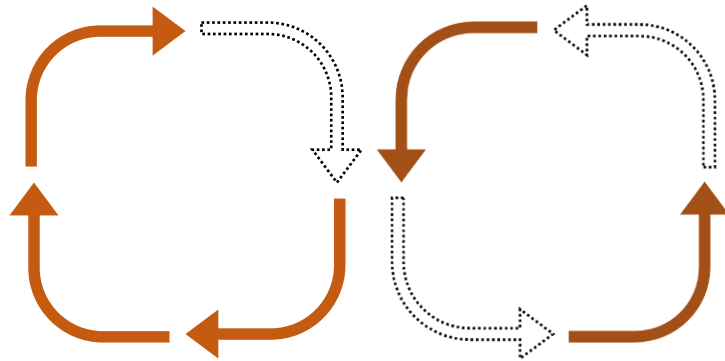


Not deadlock-free.



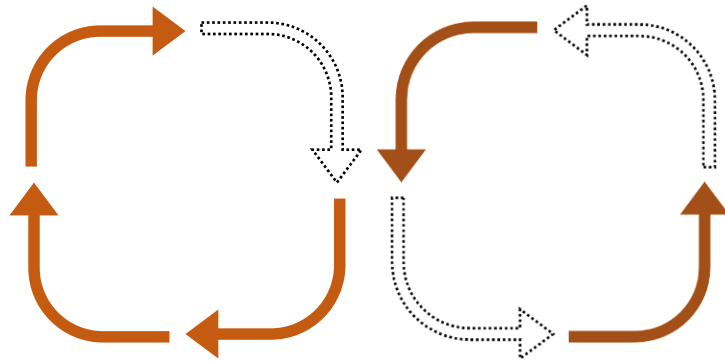
Interconnects

Consider a 4-ary 2-mesh NoC. You plan to use adaptive routing (no U-turns) with some turn restrictions to ensure deadlock-freedom. Would it be fine to disallow the following turns: E to S, S to E and N to W?

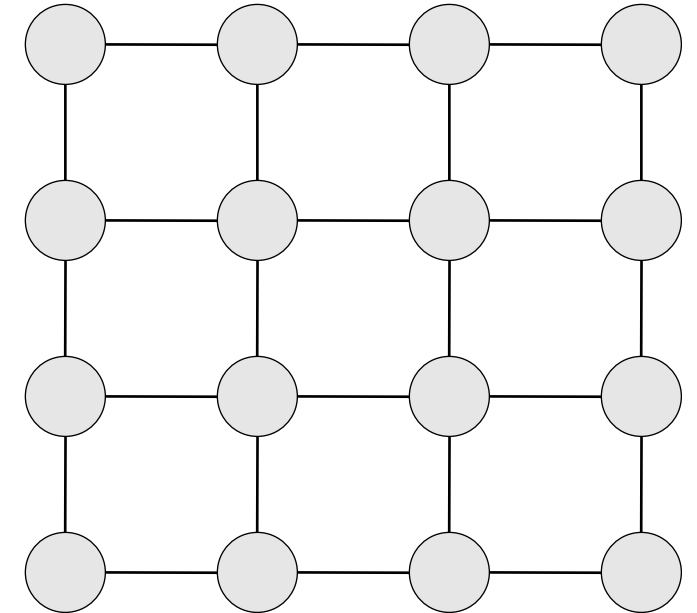


Interconnects

Consider a 4-ary 2-mesh NoC. You plan to use adaptive routing (no U-turns) with some turn restrictions to ensure deadlock-freedom. Would it be fine to disallow the following turns: E to S, S to E and N to W?

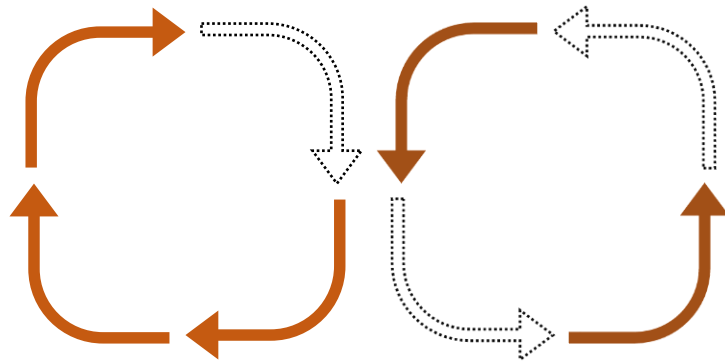


Deadlock-free...



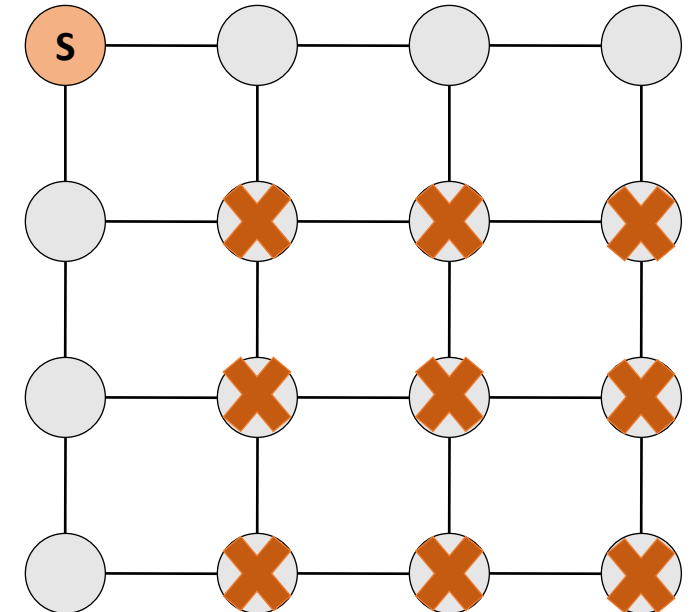
Interconnects

Consider a 4-ary 2-mesh NoC. You plan to use adaptive routing (no U-turns) with some turn restrictions to ensure deadlock-freedom. Would it be fine to disallow the following turns: E to S, S to E and N to W?



Deadlock-free...

But disallowing these turns now makes some routes impossible (i.e., SE-bound).



Interconnects

Which of the following optimizations reduce packet serialization delay?
Select all that apply.

- i. Change topology to reduce network diameter
- ii. Use larger links to increase link bandwidth
- iii. Apply crossbar speedup to increase router switch bandwidth
- iv. Shrink cache blocks to reduce number of flits per packet

Interconnects

Which of the following optimizations reduce packet serialization delay?
Select all that apply.

- i. Change topology to reduce network diameter
- ii. Use larger links to increase link bandwidth
- iii. Apply crossbar speedup to increase router switch bandwidth
- iv. Shrink cache blocks to reduce number of flits per packet

Serialization delay = # flits per packet / link bandwidth

Interconnects

Which of the following optimizations reduce packet serialization delay?
Select all that apply.

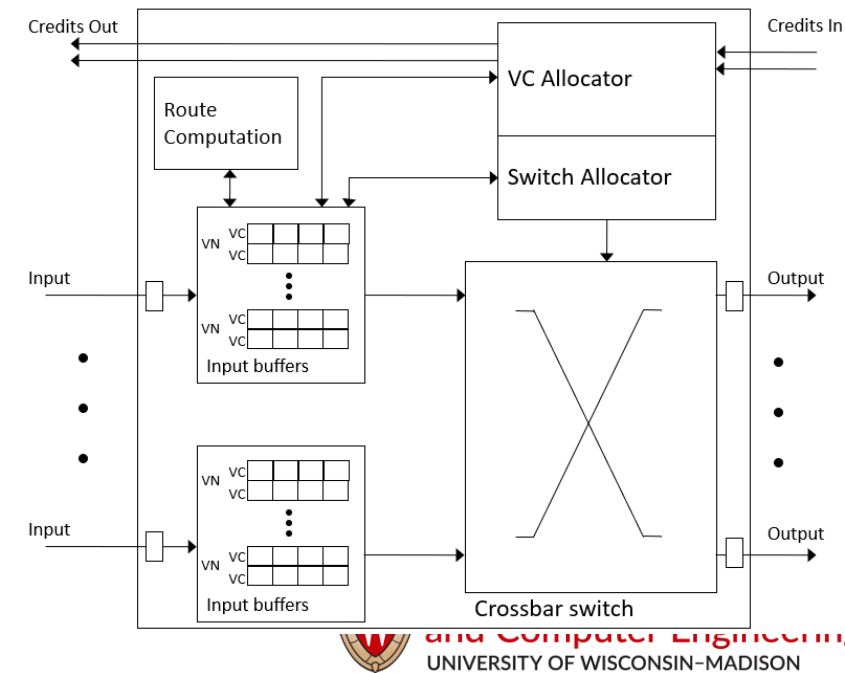
- i. Change topology to reduce network diameter
- ii. Use larger links to increase link bandwidth**
- iii. Apply crossbar speedup to increase router switch bandwidth
- iv. Shrink cache blocks to reduce number of flits per packet**

Serialization delay = # flits per packet / link bandwidth

Interconnects

Consider a router with five input (and output) ports and three virtual networks per port, each with two VCs. Assume fully adaptive routing (i.e., all turns are allowed). Fill in the numbers in the blanks:

The VC allocation stage needs _____ allocators. Each one of them allocates _____ VCs to _____ requesters (packets).

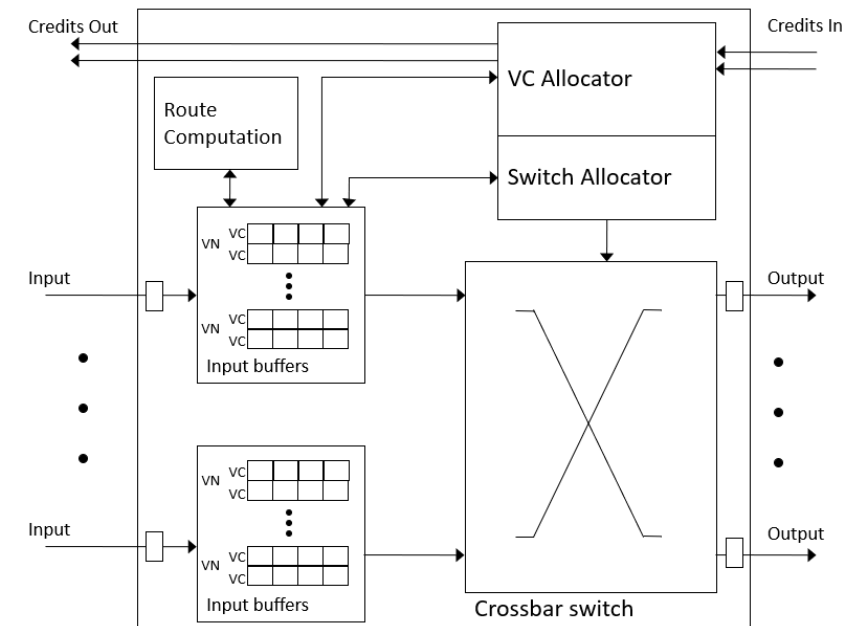


Interconnects

Consider a router with five input (and output) ports and three virtual networks per port, each with two VCs. Assume fully adaptive routing (i.e., all turns are allowed). Fill in the numbers in the blanks:

one allocator per virtual network

The VC allocation stage needs 3 allocators. Each one of them allocates _____ VCs to _____ requesters (packets).



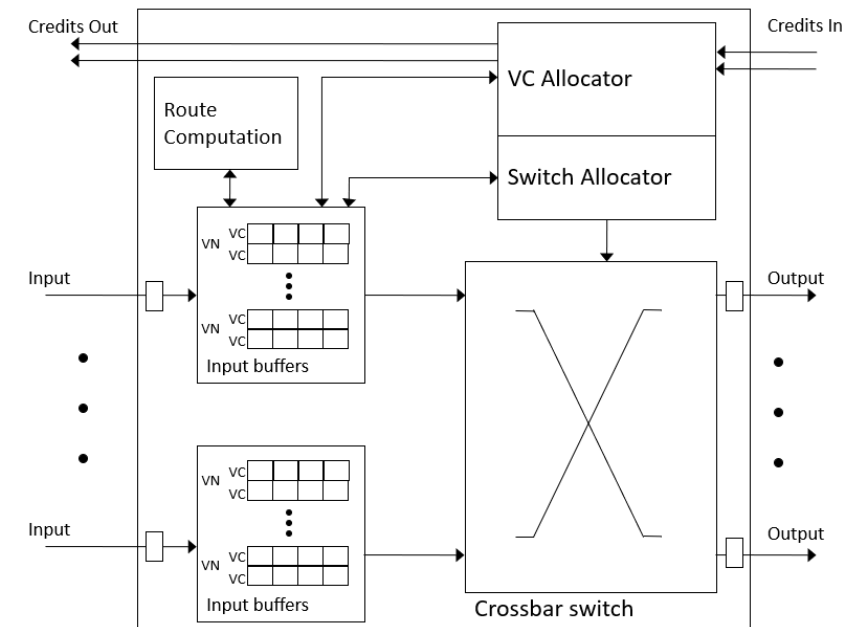
Interconnects

Consider a router with five input (and output) ports and three virtual networks per port, each with two VCs. Assume fully adaptive routing (i.e., all turns are allowed). Fill in the numbers in the blanks:

The VC allocation stage needs 3 allocators. Each one of them allocates 10 VCs to _____ requesters (packets).

five output ports with two VCs each

one allocator per virtual network



Interconnects

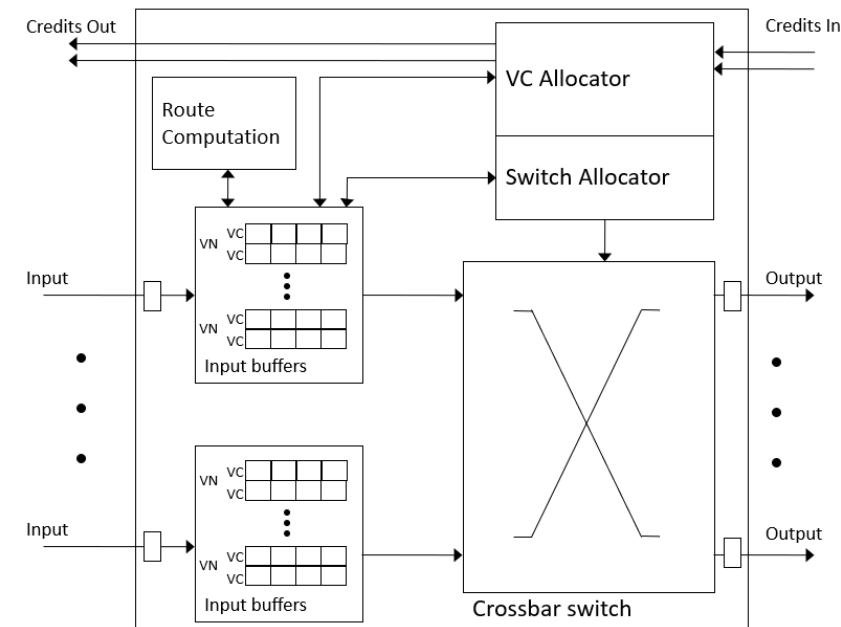
Consider a router with five input (and output) ports and three virtual networks per port, each with two VCs. Assume fully adaptive routing (i.e., all turns are allowed). Fill in the numbers in the blanks:

The VC allocation stage needs 3 allocators. Each one of them allocates 10 VCs to 10 requesters (packets).

five output ports with two VCs each

five input ports with two VCs each

one allocator per virtual network



GPUs

Threads in the same _____ may run on the same streaming multiprocessor in a GPU. Select all that apply.

- i. program
- ii. grid
- iii. block
- iv. warp

GPUs

Threads in the same _____ may run on the same streaming multiprocessor in a GPU. Select all that apply.

- i. **program**
- ii. **grid**
- iii. **block**
- iv. **warp**

GPUs

Threads in the same _____ are guaranteed to run on the same streaming multiprocessor in a GPU. Select all that apply.

- i. program
- ii. grid
- iii. block
- iv. warp

GPUs

Threads in the same _____ are guaranteed to run on the same streaming multiprocessor in a GPU. Select all that apply.

- i. program
- ii. grid
- iii. block**
- iv. warp**

GPUs

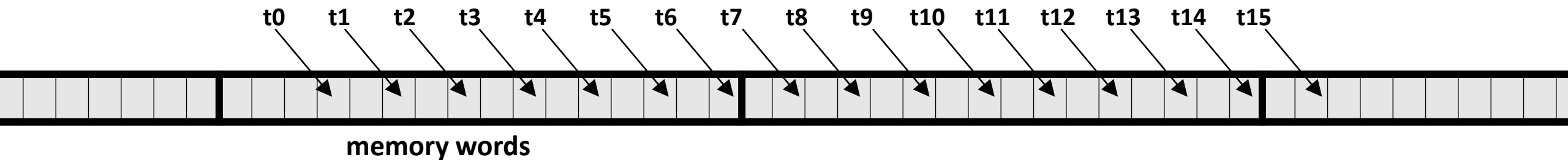
Consider the following GPU kernel. Assume 16-thread warps and 64B per memory block. At most how many cache block requests does a warp issue to read `a[]`?

```
__global__ void kernel (float *a, float *b, float c) {  
    int i = blockIdx.x * blockDim.x + threadIdx.x;  
    b[i] = a[2*i+3] * c;  
}
```

GPUs

Consider the following GPU kernel. Assume 16-thread warps and 64B per memory block. At most how many cache block requests does a warp issue to read `a[]`?

```
__global__ void kernel (float *a, float *b, float c) {  
    int i = blockIdx.x * blockDim.x + threadIdx.x;  
    b[i] = a[2*i+3] * c;  
}
```

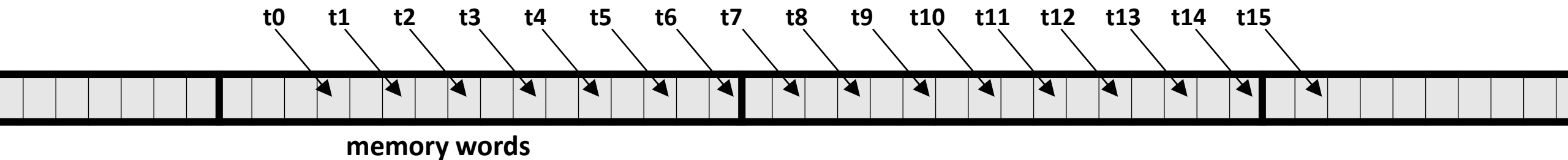


GPUs

Consider the following GPU kernel. Assume 16-thread warps and 64B per memory block. At most how many cache block requests does a warp issue to read `a[]`?

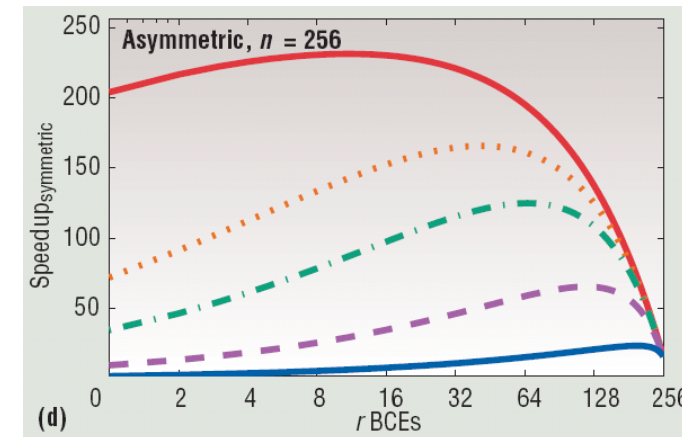
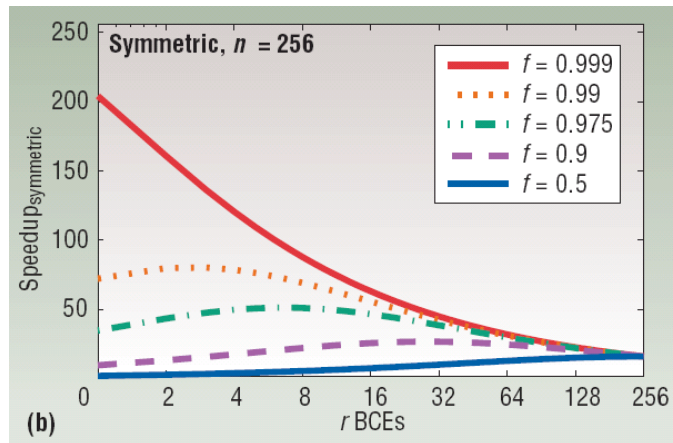
3, can't coalesce all since memory accesses are too spread out.

```
__global__ void kernel (float *a, float *b, float c) {  
    int i = blockIdx.x * blockDim.x + threadIdx.x;  
    b[i] = a[2*i+3] * c;  
}
```



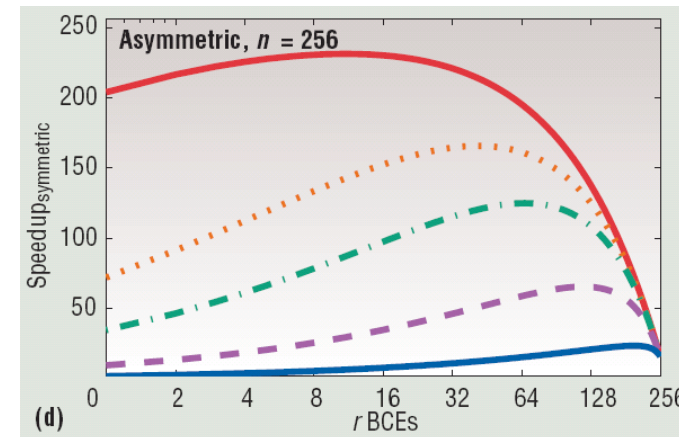
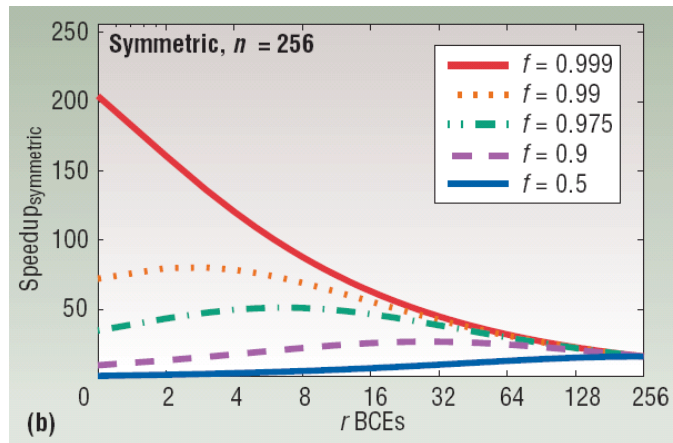
Heterogeneous Architectures and Accelerators

Applying multicore Amdahl's Law, consider a symmetric multiprocessor with all r -BCE cores vs. asymmetric with one r -BCE core and the rest 1-BCE cores. Why are the speedups on the asymmetric multicore always better?



Heterogeneous Architectures and Accelerators

Applying multicore Amdahl's Law, consider a symmetric multiprocessor with all r -BCE cores vs. asymmetric with one r -BCE core and the rest 1-BCE cores. Why are the speedups on the asymmetric multicore always better?



Only one larger r -BCE core is needed to accelerate the sequential parts of a program (and the model assumes no overheads in scheduling).

Consistency

Is the following program data-race-free? Synchronization is highlighted.

```
// P1
*x = ...;
signal(A);
lock(B);
*y += ...;
unlock(B);
wait(C);
... = *z;
... = *w;
```

```
// P2
*w = ...;
lock(B);
lock(D);
... = *v;
unlock(D);
*y += ...;
unlock(B);
signal(C);
```

```
// P3
*z = ...;
lock(B);
*y += ...;
unlock(B);
wait(A);
... = *x;
```

Consistency

Is the following program data-race-free? Synchronization is highlighted.

No, data race on z between P1 and P3

```
// P1
*x = ...;
signal(A);
lock(B);
*y += ...;
unlock(B);
wait(C);
... = *z;
... = *w;
```

```
// P2
*w = ...;
lock(B);
lock(D);
... = *v;
unlock(D);
*y += ...;
unlock(B);
signal(C);
```

```
// P3
*z = ...;
lock(B);
*y += ...;
unlock(B);
wait(A);
... = *x;
```